

Zadanie 1

Napisz funkcję generującą przyjmującą obiekt iterowalny (listę, krotkę, ...) a zwracającą generator, który zwraca elementy przekazanego obiektu ale bez *sąsiadujących* duplikatów.

Na przykład program

```
#!/usr/bin/env python
```

[download rmDupAdj.py](#)

```
def rmDupAdj(seq):  
    #  
    # ...  
    #  
  
ls = [1, 1, 4, 5, 5, 2, 1, 5, 5]  
gen = rmDupAdj(ls)  
for e in gen: print(e, end=' ')  
print('\ntype(gen):', type(gen).__name__)
```

powinien wydrukować

```
1 4 5 2 1 5  
type(gen): generator
```

Zadanie 2

Napisz funkcję generującą **slidingWindow**, która pobiera obiekt iterowalny **iterable** i liczbę naturalną **width**. Iteracja po zwracanym generatorze daje, jako krotki o długości **width**, kolejne sekwencje **width** elementów (od elementu pierwszego w pierwszej krotce, od drugiego w drugiej, itd.).

- Działanie generatora musi być „leniwe”, czyli nie może on tworzyć żadnych struktur o długości większej niż **width** na elementy do zwrócenia.
- Gdy liczba elementów w **iterable** jest mniejsza od **width**, zwracana jest jedna krotka uzupełniona wartościami **None** do pełnej długości.
- Argument **iterable** może sam być generatorem, więc nie można pobierać jego długości!

Na przykład następujący program

```
lst = [1, 2, 3, 4, 5]  
  
g = slidingWindow(lst, 3)  
print(list(g))
```

```

g = slidingWindow(lst, 5)
print(list(g))

g = slidingWindow(lst, 7)
print(list(g))

# also a generating function
def helper():
    for n in range(1, 6): yield n

g = slidingWindow(helper(), 4)
print(list(g))

```

powinien wydrukować

```

[(1, 2, 3), (2, 3, 4), (3, 4, 5)]
[(1, 2, 3, 4, 5)]
[(1, 2, 3, 4, 5, None, None)]
[(1, 2, 3, 4), (2, 3, 4, 5)]

```

Zadanie 3

Napisz funkcję generującą¹ **multimerge**, która pobiera dowolną liczbę obiektów iterowalnych, a zwrócony generator zwraca wszystkie elementy ze wszystkich obiektów iterowalnych w porządku rosnącym.²

- Zakładamy, że wszystkie obiekty iterowalne zwracają swoje elementy już posortowane (w porządku rosnącym).
- Niektóre obiekty iterowalne mogą być puste.
- Liczby elementów zwracanych przez poszczególne obiekty iterowalne może być różna i mogą być bardzo duże (a nawet nieskończone), więc scalanie danych do jednej listy nie wchodzi w grę.

UWAGA: Iteratory w Pythonie nie mają metody **hasNext**. Jeśli iterator jest już wyczerpany, wywołanie **next(iterator)** spowoduje wyjątek **StopIteration**, który możemy obsłużyć tak

```

try:
    ## ...
    e = next(iterator)
except StopIteration:
    ## ...

```

Na przykład program

¹Czyli zawierającą wyrażenie **yield**.

²Podobnie jak to jest w **merge** w algorytmie sortowania przez scalanie (*merge sort*.)

```
lst = [ [2, 3, 8], [], [0, 7], [4], [3, 4, 9, 10, 13] ]
g = multimerge(*lst)
print(list(g))
```

powinien wydrukować

```
[0, 2, 3, 3, 4, 4, 7, 8, 9, 10, 13]
```

Zadanie 4

Napisz bezargumentową funkcję generującą zwracającą korutynę, do której można (metodą **send**) posyłać liczby, a zwracane są średnie arytmetyczne wszystkich do tej pory posłanych liczb. Przesłanie **None** powoduje reset, to znaczy liczenie średniej zaczyna się od początku.

Na przykład program

```
#!/usr/bin/env python download RunningAverage.py
from functools import wraps

# decorator returning already primed coroutine
def primeIt(func):
    @wraps(func)
    def primed(*args, **kwargs):
        p = func(*args, **kwargs)
        next(p)
        return p
    return primed

@primeIt
def runningAverage():
    """
    """
    """ ...
    """

coro = runningAverage()

print( coro.send(2) )
print( coro.send(4) )
print( coro.send(6) )

print( '*resetting*' )
coro.send(None)

print( coro.send(9) )
print( coro.send(1) )
print( coro.send(8) )
```

powinien wydrukować

```
2.0
3.0
4.0
*resetting*
9.0
5.0
6.0
```

Zadanie 5

Napisz funkcję generującą³ **multi_dice**, która pobiera parametr **cnt** o domyślnej wartości 2. Zwracany generator, po „zapoczątkowaniu” (*priming*), powinien generować (teoretycznie w nieskończoność) krotki zawierające

- numer kolejny rzutu kostkami;
- krotkę zawierającą liczby oczek na **cnt** wyrzuconych kostkach do gry.⁴

Wysłanie (**send**) do generatora liczby **m** zmienia liczbę wyrzucanych w jednym rzucie kostek na **m**.

Na przykład poniższy program

```
g = multi_dice()
next(g)

for _ in range(3):
    print(next(g))

g.send(5)
for _ in range(2):
    print(next(g))

g.send(3)
for _ in range(4):
    print(next(g))
```

powinien wydrukować coś w rodzaju

```
Throwing 2 dice
(1, (5, 3))
(2, (6, 4))
(3, (4, 5))
Throwing 5 dice
(4, (5, 3, 4, 5, 2))
(5, (4, 4, 1, 1, 5))
Throwing 3 dice
```

³Czyli zawierającą wyrażenie **yield**.

⁴Do losowania liczby oczek na kostce można użyć funkcji **random.randint(1,6)**.

```
(6, (6, 5, 4))
(7, (1, 6, 1))
(8, (3, 2, 1))
(9, (2, 1, 3))
```

Zadanie 6

Napisz korutynę⁵ **rpn**, do której posyłane są dane z pewnego źródła; na przykład tak jak robi to funkcja **gen** w programie *ABW-PushPipe* z wykładu, przesyłając dane do korutyny **peek**.

Dane mają postać strumienia liczb (typu **int** lub **float**), znaków '+', '-', '*', '/' oraz napisu 'end'. Są one interpretowane jako składniki wyrażeń w Odwrotnej Notacji Polskiej (RPN). Każde pojedyncze wyrażenie kończy się przesłaniem napisu 'end'.

Korutyna **rpn** otrzymuje te dane i po odebraniu napisu 'end' przesyła wynik do korutyny **toList**, która te wyniki zbiera do listy.

Na przykład program

```
src = [ 5, 7, '+', 2, '*', 20, '-', 'end',
        7, 2, '-', 2, 1, '+', '/', 3, '*', 5, '+', 'end',
        5, 2, '-', 2.5, 2, '*', '*', 13.5, '-', 'end' ]
lst = []
gen(src, rpn(toList(lst)))
print('Results:', lst)
```

powinien wydrukować

```
Results: [4, 10.0, 1.5]
```

Do realizacji stosu można użyć zwykłej listy z jej metodami **pop** i **append**.

Dla operacji niesymetrycznych (odejmowanie i dzielenie) zakładamy, że prawy operand jest zdejmowany ze stosu przed lewym.

⁵Jak na przykład w programie *ABW-PushPipe* z wykładu — *nie* chodzi tu o korutyny z pakietu *asyncio*!